

ERAASOFT

Task 20

Osama Sabry Fekry

Part 1 – Understanding argsort()

numpy.argsort() a function that returns the indices that would sort an array, it gives indices that will be used to reorder the elements in an array to be in sorted order. For example:

```
import numpy as np

arr = np.array([30, 10, 50, 20, 40])

indices = np.argsort(arr)

print('Indices:', indices)

print('Values:', arr[indices])
```

Output

```
Indices: [1 3 0 4 2]

Values: [10 20 30 40 50]
```

Syntax and parameters

The practical syntax is np.argsort(a, axis=-1, kind=None, order=None).

Parameter	Meaning
a	The array or list to sort.
axis	Default -1 means the last axis. For 2D data, 0 sorts columns and 1 sorts rows. None uses flattened positions.
kind	Sorting algorithm: quicksort, mergesort, heapsort or stable. Default uses quicksort.
order	Field order for a structured array, such as order="age". Unspecified fields can break ties.
stable	Optional in NumPy 2.0 and later. True keeps equal keys in their starting order. kind="stable" also requests this.
descending	Available in NumPy 2.5 and later. True requests descending order.

Return value

The return value is an integer NumPy array. Its shape matches the input when an axis is specified. For axis=None, it is a one-dimensional array of positions in the flattened input.

```
import numpy as np

arr = np.array([8, 3, 6])

indices = np.argsort(arr)

print('Indices:', indices)

print('Shape:', indices.shape)

print('Is integer:', np.issubdtype(indices.dtype, np.integer))
```

Output

```
Indices: [1 2 0]

Shape: (3,)

Is integer: True
```

Difference between sort() and argsort()

np.sort(arr) returns a sorted copy of the values. np.argsort(arr) returns positions. The method arr.sort() changes arr directly and returns None.

```
import numpy as np

arr = np.array([8, 3, 6])

print('np.sort:', np.sort(arr))

print('np.argsort:', np.argsort(arr))

print('Original:', arr)

result = arr.sort()

print('After arr.sort:', arr)

print('Return from arr.sort:', result)
```

Output

```
np.sort: [3 6 8]

np.argsort: [1 2 0]

Original: [8 3 6]

After arr.sort: [3 6 8]

Return from arr.sort: None
```

Sorting in ascending and descending order

argsort() sorts ascending by default. For a one-dimensional array of finite numbers, reversing its indices gives descending order. Reversing also reverses equal-value ties. NaN values need separate handling because they move to the front when reversed.

```
import numpy as np

arr = np.array([8, 3, 6])

ascending = np.argsort(arr)

descending = ascending[::-1]

print('Ascending indices:', ascending)

print('Ascending values:', arr[ascending])

print('Descending indices:', descending)

print('Descending values:', arr[descending])
```

Output

```
Ascending indices: [1 2 0]

Ascending values: [3 6 8]

Descending indices: [0 2 1]

Descending values: [8 6 3]
```

Sorting along different axes (axis=0 and axis=1)

In a two-dimensional array, axis=1 sorts across each row and axis=0 sorts down each column. Use take_along_axis() to apply the positions along the same axis. Each row or column is sorted independently.

```
import numpy as np

arr = np.array([[9, 2, 7], [4, 8, 1]])

row_indices = np.argsort(arr, axis=1)

column_indices = np.argsort(arr, axis=0)

print('Row indices:\n', row_indices)

print('Sorted rows:\n', np.take_along_axis(arr, row_indices, axis=1))

print('Column indices:\n', column_indices)

print('Sorted columns:\n', np.take_along_axis(arr, column_indices, axis=0))
```

Output

```
Row indices:

 [[1 2 0]

 [2 0 1]]

Sorted rows:

 [[2 7 9]

 [1 4 8]]

Column indices:

 [[1 0 1]

 [0 1 0]]

Sorted columns:

 [[4 2 1]

 [9 8 7]]
```

Part 2 – Working with argsort()

Sorting a one-dimensional array using argsort()

Get the positions in ascending order, then use them to select the values.

```
import numpy as np

arr = np.array([30, 10, 50, 20, 40])

indices = np.argsort(arr)

print(arr[indices])
```

Output

```
[10 20 30 40 50]
```

Sorting an array in descending order using argsort()

Reverse the sorted positions before selecting the values.

```
import numpy as np

arr = np.array([30, 10, 50, 20, 40])

indices = np.argsort(arr)[::-1]

print(arr[indices])
```

Output

```
[50 40 30 20 10]
```

Sorting each row of a two-dimensional array

`axis=1` works across columns inside each row. The row boundaries stay in place.

```
import numpy as np

arr = np.array([[9, 2, 7], [4, 8, 1]])

indices = np.argsort(arr, axis=1)

result = np.take_along_axis(arr, indices, axis=1)

print('Indices:\n', indices)
```

```
print('Result:\n', result)
```

Output

```
Indices:
 [[1 2 0]
 [2 0 1]]
Result:
 [[2 7 9]
 [1 4 8]]
```

Sorting each column of a two-dimensional array

axis=0 works down the rows inside each column. This sorts columns independently, so it does not preserve complete row records.

```
import numpy as np
arr = np.array([[9, 2, 7], [4, 8, 1]])
indices = np.argsort(arr, axis=0)
result = np.take_along_axis(arr, indices, axis=0)
print('Indices:\n', indices)
print('Result:\n', result)
```

Output

```
Indices:
 [[1 0 1]
 [0 1 0]]
Result:
 [[4 2 1]
 [9 8 7]]
```

Rearranging an array using the indices returned by argsort()

Use the same positions for related arrays. This keeps each name paired with its value.

```
import numpy as np

names = np.array(['A', 'B', 'C'])

values = np.array([30, 10, 20])

indices = np.argsort(values)

print('Indices:', indices)

print('Rearranged names:', names[indices])

print('Rearranged values:', values[indices])
```

Output

```
Indices: [1 2 0]

Rearranged names: ['B' 'C' 'A']

Rearranged values: [10 20 30]
```

Outputs of sort() and argsort() Comparison

Sort() gives values; argsort() gives positions. Applying those positions reproduces the sorted values.

```
import numpy as np

arr = np.array([30, 10, 50, 20, 40])

indices = np.argsort(arr)

print('sort:', np.sort(arr))

print('argsort:', indices)

print('arr[indices]:', arr[indices])
```

Output

```
sort: [10 20 30 40 50]

argsort: [1 3 0 4 2]

arr[indices]: [10 20 30 40 50]
```

Part 3 Applications of argsort()

Finding the indices of the smallest and largest values.

Problem: identify the original positions of the minimum and maximum. The first sorted position is the minimum; the last is the maximum.

```
import numpy as np

arr = np.array([30, 10, 50, 20, 40])

indices = np.argsort(arr)

print('Smallest index:', indices[0])

print('Smallest value:', arr[indices[0]])

print('Largest index:', indices[-1])

print('Largest value:', arr[indices[-1]])
```

Output

```
Smallest index: 1

Smallest value: 10

Largest index: 2

Largest value: 50
```

Finding the Top-K largest elements.

Problem: select the three largest values. Reverse the sorted positions, then take the first K. K is assumed to be between zero and the array length.

```
import numpy as np

arr = np.array([30, 10, 50, 20, 40])

k = 3

indices = np.argsort(arr)[::-1][:k]

print('Indices:', indices)

print('Largest values:', arr[indices])
```

Output

Indices: [2 4 0]

Largest values: [50 40 30]

Finding the Top-K smallest elements.

Problem: select the three smallest values. Take the first K ascending positions.

```
import numpy as np

arr = np.array([30, 10, 50, 20, 40])

k = 3

indices = np.argsort(arr)[:k]

print('Indices:', indices)

print('Smallest values:', arr[indices])
```

Output

Indices: [1 3 0]

Smallest values: [10 20 30]

Ranking students according to their grades.

Problem: list students from highest grade to lowest. Sort students from highest grade to lowest. If students have the same grade, keep them in their original order. The printed ranks are list positions, with tied students listed consecutively.

```
import numpy as np

students = np.array(['Ali', 'Mona', 'Omar', 'Sara'])

grades = np.array([75, 90, 85, 90], dtype=int8)

indices = np.argsort(-grades, kind='stable')

for rank, index in enumerate(indices, start=1):

    print(rank, students[index], int(grades[index]))
```

Output

1 Mona 90

2 Sara 90

3 Omar 85

4 Ali 75

Ranking products according to their prices.

Problem: list products from cheapest to most expensive. Apply the sorted price positions to both prices and product names.

```
import numpy as np

products = np.array(['Mouse', 'Keyboard', 'Monitor'])

prices = np.array([350, 800, 5000])

indices = np.argsort(prices)

for rank, index in enumerate(indices, start=1):

    print(rank, products[index], prices[index])
```

Output

1 Mouse 350

2 Keyboard 800

3 Monitor 5000

Sorting one array based on another array.

Problem: arrange names by age. Sort ages, then apply the resulting positions to both arrays. The same index must represent the same person in each input array.

```
import numpy as np

names = np.array(['Ali', 'Mona', 'Omar'])

ages = np.array([22, 19, 21])

indices = np.argsort(ages)

print('Names:', names[indices])

print('Ages:', ages[indices])
```

Output

```
Names: ['Mona' 'Omar' 'Ali']

Ages: [19 21 22]
```

Sorting records while preserving their original indices.

Problem: sort student records by grade while keeping their starting row numbers. Sort the numeric grade column, then select whole rows. This preserves complete records and their original positions.

```
import numpy as np

records = np.array([['Ali', '75'], ['Mona', '90'], ['Omar', '85']])

grades = records[:, 1].astype(float)

indices = np.argsort(grades)

for original_index in indices:

    print('Original row:', original_index, 'Record:', records[original_index])
```

Output

```
Original row: 0 Record: ['Ali' '75']

Original row: 2 Record: ['Omar' '85']

Original row: 1 Record: ['Mona' '90']
```

Retrieving the original positions of sorted values.

Problem: show where every sorted value came from. Keep the argsort() result and pair each sorted value with its source index. This also works when values repeat, unlike searching for each value by equality.

```
import numpy as np

arr = np.array([40, 10, 40, 20])

indices = np.argsort(arr, kind='stable')

sorted_values = arr[indices]

for i in range(len(arr)):

    print('Value:', sorted_values[i], 'Original index:', indices[i])

# The inverse permutation restores the initial order.

inverse = np.argsort(indices)

print('Restored:', sorted_values[inverse])
```

Output

```
Value: 10 Original index: 1

Value: 20 Original index: 3

Value: 40 Original index: 0

Value: 40 Original index: 2

Restored: [40 10 40 20]
```

Part 4 – Comparative Analysis

We use sort() when only the values are needed, argsort() when original positions matter, and lexsort() when several keys determine the order. These functions return new arrays. The separate method arr.sort() updates the input in place and returns None.

Feature	np.sort()	np.argsort()	np.lexsort()
Purpose	Sort values	Sort positions for one key	Sort positions using multiple keys
Return value	Sorted copy of the values	Integer index array	Integer index array
Typical use	An ordered list of numbers	Ranks, top K and paired arrays	Category then price, or surname then first name
Advantage	Direct and easy to read	Keeps the connection to original positions	Handles several keys and preserves equal-key order
Limitation	Does not return original positions	Needs indexing to obtain values; full sorting costs work	Keys must have matching shapes; last key must be remembered

Conclusion

Keeping the argsort() indices is useful whenever sorted data must stay connected to names, records or original rows. Use the same indices for related arrays. For two-dimensional data, apply the indices along the axis that was sorted. A stable sort keeps equal-key items in their original order. Full argsort() is easy to understand for top K; for very large arrays, a partial selection can reduce work. Always check the dtype, shape and indexing direction before sorting.